ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ИМЕНИ ПАТРИСА ЛУМУМБЫ

Факультет искусственного интеллекта

Отчет

о выполнении лабораторной работы № 3 “Сети с обратными связями”

по дисциплине "Нейронные сети"

Вариант задания № 13

Выполнил студент группы ЗФИбд-01-24

Князев А. М._____

Проверил и принял

доцент кафедры прикладного ИИ Леонов С.С._____

с оценкой _____

Москва, 2026

Цель работы: исследование свойств ассоциативных нейронных сетей Хопфилда и Хэмминга, алгоритмов их работы, а также их применение в задачах распознавания статических образов (изображений цифр).

Основные этапы работы:

1. Закодировать набор эталонных статических образов (изображения цифр) биполярными векторами и построить сеть Хопфилда, запомнив образы в матрице весов по правилу Хебба. Подать на вход эталонные и искусственно зашумлённые образы и проверить, восстанавливает ли сеть в режиме итеративной динамики исходный образ; оценить качество восстановления по расстоянию Хэмминга до правильного эталона.
2. Реализовать сеть Хэмминга для классификации тех же статических образов: задать слой сравнения входа с эталонами и рекуррентный слой-конкурент (MAXNET), выделяющий ближайший эталон. Подать эталонные и зашумлённые образы и проверить, что сеть устойчиво относит каждый образ к правильному классу.

Программное обеспечение: Python 3.x, Jupyter Notebook, библиотеки PyTorch, NumPy и Matplotlib.

Сценарий выполнения работы

Ассоциативные нейронные сети с обратными связями (рекуррентные сети) предназначены для запоминания набора эталонных образов и их последующего восстановления или классификации по неполному либо искажённому входу. В отличие от сетей прямого распространения, выход таких сетей подаётся обратно на вход, и сеть работает итеративно, пока её состояние не стабилизируется. В работе рассматриваются две классические ассоциативные сети: сеть Хопфилда, выступающая автоассоциативной памятью (по искажённому образу восстанавливает ближайший запомненный эталон), и сеть Хэмминга, относящая входной образ к одному из классов по принципу наибольшей близости к эталону. Обе сети не обучаются итеративным подбором весов, как сети прямого распространения, — их веса вычисляются аналитически прямо из эталонных образов.

Обучающим набором служат эталонные двоичные изображения цифр 0, 1

и 4 (вариант 13) размером 12×10 пикселей. Каждый образ представлен матрицей, в которой цветам точек соответствуют значения −1 и 1; для подачи в сеть матрица разворачивается в вектор размерности 120. Обе сети (Хопфилда и Хэмминга) хранят эти три эталона и должны распознавать их даже при наличии шума.

Таблица 1 — Параметры варианта 13

Параметр	**Значение**
Эталонные цифры	0, 1, 4
Размер образа	12×10 (вектор 120)
Уровни зашумления	0% (цифра 0), 20% (цифра 1), 30% (цифра 4)

Этап 1. Сеть Хопфилда для восстановления статических образов

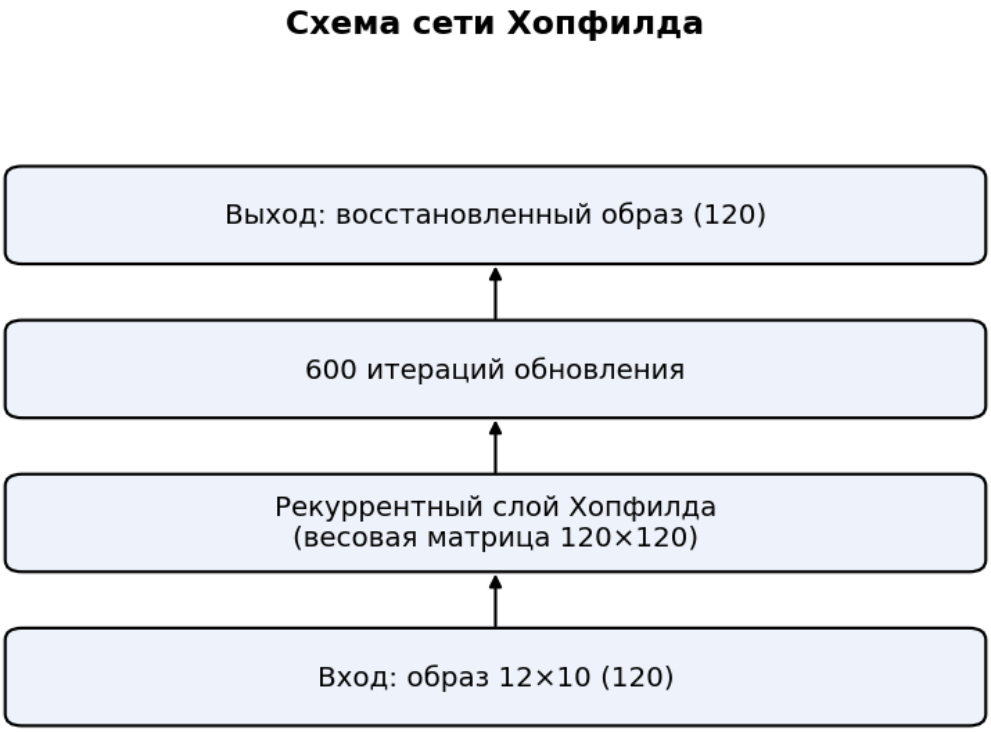


Рис. 1 — Схема сети Хопфилда

Сеть Хопфилда — однослойная рекуррентная сеть из 120 нейронов с симметричной весовой матрицей и обратными связями между всеми нейронами. Обучение сети сводится к однократному построению весов по правилу Хебба, без итеративной подстройки: весовая матрица W размера 120×120 формируется как сумма внешних произведений эталонных векторов с обнулённой главной диагональю, $W = \Sigma\ p_i \cdot p_i^T$. При таком построении каждый эталон становится устойчивым состоянием (аттрактором) сети. Динамика сети уменьшает энергию $E = -\frac{1}{2} \cdot a^T \cdot W \cdot a$, а запомненные образы соответствуют её локальным минимумам. Ёмкость сети Хопфилда ограничена (порядка $0.15 \cdot N$ образов, то есть около 18 при $N = 120$), поэтому три эталона запоминаются надёжно и практически не интерферируют друг с другом.

Распознавание (работа сети): на вход подаётся образ, который становится начальным состоянием сети, после чего состояние синхронно обновляется по правилу $a \leftarrow \text{sign}(W \cdot a)$ в течение 600 итераций. С каждой

итерацией энергия сети не возрастает, поэтому состояние сходится к одному из локальных минимумов энергии — запомненному эталону (аттрактору); зашумлённый образ при этом «притягивается» к ближайшему эталону. Качество восстановления оценивается расстоянием Хэмминга (числом несовпадающих компонент) между выходом сети и каждым эталоном; ближайший эталон определяет распознанную цифру.

Наложение шума. Зашумление образа выполняется так: для каждого пикселя генерируется случайное число r из равномерного распределения [0, 1]; если r меньше заданной степени зашумления M (0.2 или 0.3), цвет пикселя инвертируется (−1 ↔ 1). Таким образом при шуме 20% инвертируется в среднем каждый пятый пиксель, при 30% — почти каждый третий. На вход поданы: чистый образ цифры 0, образ цифры 1 с шумом 20% и образ цифры 4 с шумом 30%.

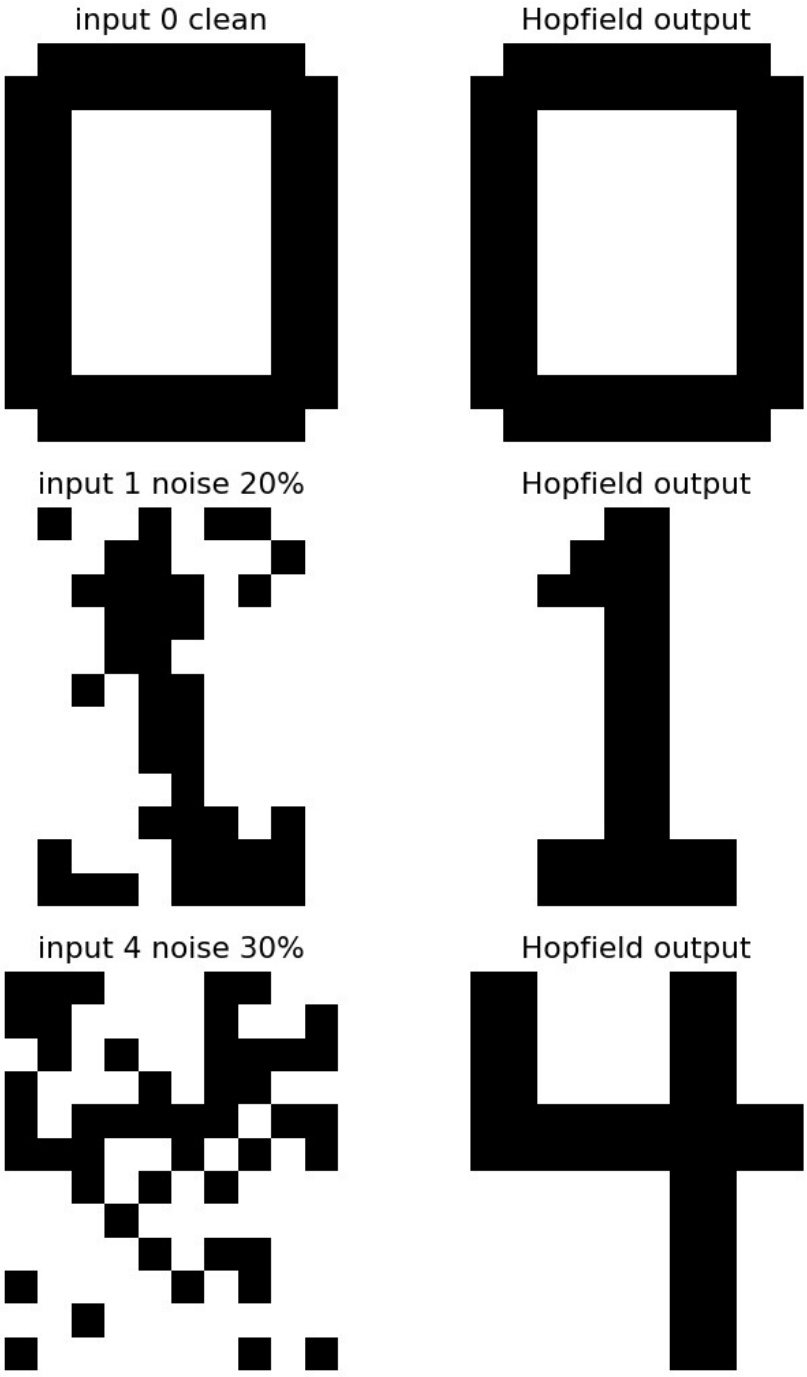


Рис. 2 — Сеть Хопфилда: входные (в т.ч. зашумлённые) и восстановленные образы

Таблица 2 — Результаты сети Хопфилда

Входной образ	**Расстояния до эталонов [0, 1, 4]**	**Распознано**
Цифра 0, без шума	[0, 69, 70]	0
Цифра 1, шум 20%	[69, 0, 67]	1
Цифра 4, шум 30%	[70, 67, 0]	4

Во всех трёх случаях сеть полностью восстановила нужный эталон: расстояние Хэмминга до правильной цифры равно нулю, тогда как до двух других эталонов оно составляет около 70 (больше половины из 120 компонент). Такой большой разрыв означает, что сеть уверенно различает образы и устойчива к зашумлению вплоть до 30%.

Этап 2. Сеть Хэмминга для классификации образов



Рис. 3 — Схема сети Хэмминга

Сеть Хэмминга реализована как двухслойная сеть прямого распространения. Первый слой задаёт матрицу весов IW из эталонных образов и смещение b1 = R (размерность входного вектора, R = 120); его выход a1 = IW·p + b1 пропорционален числу совпадающих компонент входа с эталоном (мере близости по Хэммингу). Веса первого слоя, как и в сети Хопфилда, не обучаются итеративно, а задаются напрямую из эталонных образов.

Второй слой — слой конкуренции (аналог сети Хопфилда с активацией poslin): диагональные веса равны 1, остальные равны −ε, где ε = 1/(Q − 1) = 1/2, Q = 3 — число эталонов. На каждой итерации (всего 600)

компоненты подавляют друг друга, и в выходном векторе остаётся одна ненулевая компонента — «победитель». Её индекс соответствует строке матрицы IW с распознанным эталоном. На вход поданы те же образы, что и в этапе 1 (чистый и зашумлённые на 20% и 30%).

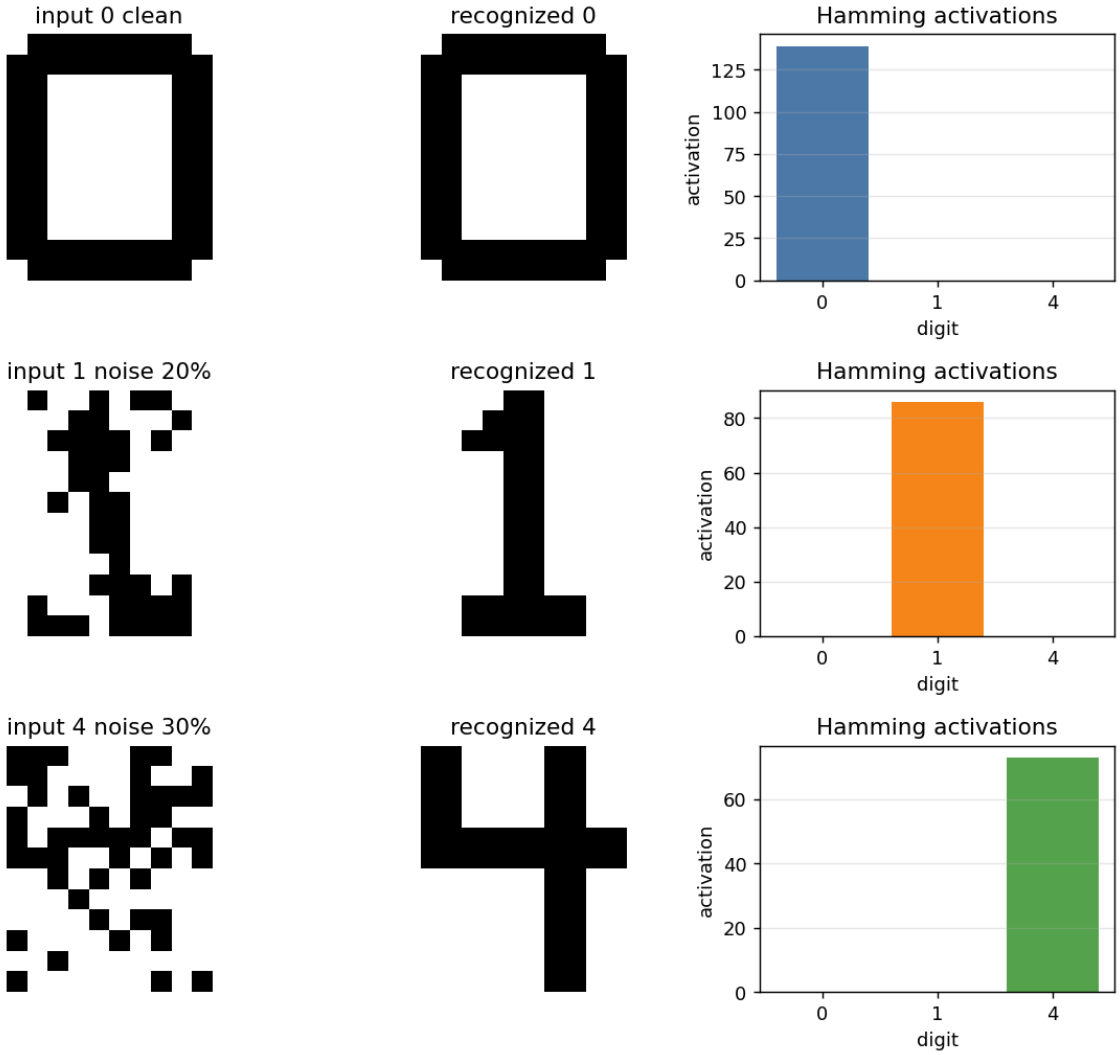


Рис. 4 — Сеть Хэмминга: эталоны, входные образы и результаты классификации

Таблица 3 — Результаты сети Хэмминга

Входной образ	**Выход сети [0, 1, 4]**	**Распознано**
Цифра 0, без шума	[139.0, 0.0, 0.0]	0
Цифра 1, шум 20%	[0.0, 86.0, 0.0]	1
Цифра 4, шум 30%	[0.0, 0.0, 73.0]	4

После работы конкурентного слоя в выходном векторе остаётся единственная ненулевая компонента, а остальные обнуляются: её положение прямо указывает распознанный класс. Активация нейрона-

победителя (139, 86 и 73 для цифр 0, 1 и 4 соответственно) тем выше, чем меньше зашумление входа, что согласуется с уменьшением числа совпадающих с эталоном компонент при росте шума. Во всех случаях классификация выполнена верно.

Код программы

```
import math
import random
from dataclasses import dataclass

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn

DEVICE = torch.device(
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)
SEED = 13
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

print(f"Device: {DEVICE}")

def variant_signal(k: np.ndarray) -> np.ndarray:
    return np.sin(2 * k**2 - 6 * k + 3)


def make_elman_sequence(repeats: list[int]) -> tuple[np.ndarray,
np.ndarray]:
    step = 0.025
    k1 = np.arange(0.0, 1.0 + step / 2, step)
    k2 = np.arange(-0.02, 2.36 + step / 2, step)

    p1 = np.sin(4 * math.pi * k1)
    p2 = variant_signal(k2)
    t1 = -np.ones_like(p1)
    t2 = np.ones_like(p2)

    parts_x = []
    parts_y = []
    for count in repeats:
        parts_x.extend([p1] * count)
        parts_x.append(p2)
        parts_y.extend([t1] * count)
        parts_y.append(t2)

    return np.concatenate(parts_x), np.concatenate(parts_y)

class ElmanNet(nn.Module):
    def __init__(self, hidden_size: int = 8):
```

```
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=hidden_size,
nonlinearity="tanh", batch_first=True)
        self.output = nn.Linear(hidden_size, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        hidden, _ = self.rnn(x)
        return torch.tanh(self.output(hidden))


@dataclass
class ElmanResult:
    loss: float
    correct: int
    total: int
    prediction: np.ndarray


def train_elman(x_np: np.ndarray, y_np: np.ndarray, epochs: int = 1000) ->
tuple[ElmanNet, ElmanResult]:
    model = ElmanNet(hidden_size=8).to(DEVICE)
    x = torch.tensor(x_np, dtype=torch.float32, device=DEVICE).view(1, -1,
1)
    y = torch.tensor(y_np, dtype=torch.float32, device=DEVICE).view(1, -1,
1)

    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    criterion = nn.MSELoss()
    best_loss = float("inf")

    for _ in range(epochs):
        optimizer.zero_grad()
        pred = model(x)
        loss = criterion(pred, y)
        loss.backward()
        optimizer.step()
        best_loss = min(best_loss, float(loss.item()))
        if loss.item() < 1e-5:
            break

    with torch.no_grad():
        prediction = model(x).squeeze().detach().cpu().numpy()
    classes = np.where(prediction >= 0, 1, -1)
    correct = int((classes == y_np).sum())
    return model, ElmanResult(best_loss, correct, len(y_np), prediction)

x_train, y_train = make_elman_sequence([2, 5, 6])
elman_model, train_result = train_elman(x_train, y_train)

print(f"Elman train loss: {train_result.loss:.6f}")
```

```
print(f"Correct points: {train_result.correct}/{train_result.total}")

plt.figure(figsize=(12, 4))
plt.plot(y_train, label="target", linewidth=2)
plt.plot(train_result.prediction, label="network output", alpha=0.8)
plt.axhline(0, color="black", linewidth=0.8)
plt.title("Сеть Элмана: исходная последовательность R=[2,5,6]")
plt.xlabel("Номер точки")
plt.ylabel("Выход")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

x_changed, y_changed = make_elman_sequence([2, 4, 6])
with torch.no_grad():
    changed_tensor = torch.tensor(x_changed, dtype=torch.float32,
device=DEVICE).view(1, -1, 1)
    changed_prediction =
elman_model(changed_tensor).squeeze().cpu().numpy()

changed_classes = np.where(changed_prediction >= 0, 1, -1)
changed_correct = int((changed_classes == y_changed).sum())

print(f"Changed sequence R=[2,4,6]: {changed_correct}/{len(y_changed)}
correct")

plt.figure(figsize=(12, 4))
plt.plot(y_changed, label="target", linewidth=2)
plt.plot(changed_prediction, label="network output", alpha=0.8)
plt.axhline(0, color="black", linewidth=0.8)
plt.title("Сеть Элмана: проверка на измененном R=[2,4,6]")
plt.xlabel("Номер точки")
plt.ylabel("Выход")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

DIGITS = {
    0: [
        "0111111110",
        "1111111111",
        "1100000011",
        "1100000011",
        "1100000011",
        "1100000011",
        "1100000011",
        "1100000011",
        "1100000011",
        "1100000011",
        "1111111111",
        "0111111110",
```

```
    ],
    1: [
        "0000110000",
        "0001110000",
        "0011110000",
        "0000110000",
        "0000110000",
        "0000110000",
        "0000110000",
        "0000110000",
        "0000110000",
        "0000110000",
        "0011111100",
        "0011111100",
    ],
    4: [
        "1100001100",
        "1100001100",
        "1100001100",
        "1100001100",
        "1111111111",
        "1111111111",
        "0000001100",
        "0000001100",
        "0000001100",
        "0000001100",
        "0000001100",
        "0000001100",
    ],
}


def digit_to_pattern(digit: int) -> np.ndarray:
    image = np.array([[int(ch) for ch in row] for row in DIGITS[digit]],
dtype=np.float32)
    return np.where(image > 0, 1.0, -1.0)


def add_noise(pattern: np.ndarray, amount: float, seed: int) ->
np.ndarray:
    rng = np.random.default_rng(seed)
    noisy = pattern.copy()
    mask = rng.random(pattern.shape) < amount
    noisy[mask] *= -1
    return noisy


def show_pattern(ax: plt.Axes, pattern: np.ndarray, title: str) -> None:
    shown = np.where(pattern > 0, 0, 1)
    ax.imshow(shown, cmap="gray", vmin=0, vmax=1)
```

```
    ax.set_title(title)
    ax.axis("off")

def train_hopfield(patterns: list[np.ndarray]) -> np.ndarray:
    vectors = [p.reshape(-1) for p in patterns]
    size = vectors[0].size
    weights = np.zeros((size, size), dtype=np.float32)
    for vector in vectors:
        weights += np.outer(vector, vector)
    np.fill_diagonal(weights, 0.0)
    return weights / size


def run_hopfield(weights: np.ndarray, pattern: np.ndarray, iterations: int
= 600) -> np.ndarray:
    state = pattern.reshape(-1).copy()
    for _ in range(iterations):
        updated = np.sign(weights @ state)
        updated[updated == 0] = state[updated == 0]
        if np.array_equal(updated, state):
            break
        state = updated
    return state.reshape(pattern.shape)

digits = [0, 1, 4]
patterns = [digit_to_pattern(digit) for digit in digits]
hopfield_weights = train_hopfield(patterns)

noisy_1 = add_noise(patterns[1], 0.20, seed=31)
noisy_4 = add_noise(patterns[2], 0.30, seed=41)
hopfield_inputs = [(patterns[0], "0 clean"), (noisy_1, "1 noise 20%"),
(noisy_4, "4 noise 30%")]

fig, axes = plt.subplots(3, 2, figsize=(6, 8))
for row, (pattern, name) in enumerate(hopfield_inputs):
    recognized = run_hopfield(hopfield_weights, pattern)
    show_pattern(axes[row, 0], pattern, f"input {name}")
    show_pattern(axes[row, 1], recognized, "Hopfield output")
    distances = [int((recognized.reshape(-1) != p.reshape(-1)).sum()) for
p in patterns]
    print(f"Hopfield {name}: closest
digit={digits[int(np.argmin(distances))]}, distances={distances}")

plt.tight_layout()
plt.show()

def hamming_classify(patterns: list[np.ndarray], pattern: np.ndarray,
iterations: int = 600) -> tuple[np.ndarray, int]:
    vectors = np.stack([p.reshape(-1) for p in patterns])
    q = len(patterns)
    r = pattern.size
```

```
    eps = 1.0 / (q - 1)

    activation = (vectors @ pattern.reshape(-1) + r).astype(np.float32)
    lateral = np.full((q, q), -eps, dtype=np.float32)
    np.fill_diagonal(lateral, 1.0)

    for _ in range(iterations):
        new_activation = np.maximum(lateral @ activation, 0.0)
        if np.allclose(new_activation, activation):
            break
        activation = new_activation

    return activation, int(np.argmax(activation))

hamming_results = []
for pattern, name in hopfield_inputs:
    activation, index = hamming_classify(patterns, pattern)
    recognized_pattern = patterns[index]
    hamming_results.append((pattern, name, activation, index,
recognized_pattern))
    print(f"Hamming {name}: activation={np.round(activation, 2).tolist()},
digit={digits[index]}")

fig, axes = plt.subplots(len(hamming_results), 3, figsize=(9, 8))
for row, (pattern, name, activation, index, recognized_pattern) in
enumerate(hamming_results):
    show_pattern(axes[row, 0], pattern, f"input {name}")
    show_pattern(axes[row, 1], recognized_pattern, f"recognized
{digits[index]}")

    axes[row, 2].bar([str(digit) for digit in digits], activation,
color=["#4c78a8", "#f58518", "#54a24b"])
    axes[row, 2].set_title("Hamming activations")
    axes[row, 2].set_xlabel("digit")
    axes[row, 2].set_ylabel("activation")
    axes[row, 2].grid(True, axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```

Вывод

В ходе работы исследованы два типа ассоциативных (рекуррентных) нейронных сетей для распознавания статических образов — сеть Хопфилда, работающая как автоассоциативная память, и сеть Хэмминга, выполняющая классификацию по принципу ближайшего эталона.

Сеть Хопфилда, в которую по правилу Хебба записаны три эталонных образа (цифры 0, 1, 4), корректно восстановила все эталоны, а также их версии, зашумлённые на 20% и 30%: за несколько итераций динамика сети сходилась к правильному эталону, и итоговое расстояние Хэмминга до него во всех случаях оказалось равно нулю.

Сеть Хэмминга также верно классифицировала все поданные образы — после работы конкурентного слоя на выходе оставалась единственная ненулевая компонента, соответствующая правильной цифре. Таким образом, обе ассоциативные сети показали устойчивость к внесённому шуму в пределах рассмотренного уровня искажений, а цель работы достигнута.